DESIGN DOC PRESENTATION

Group 7 - Psychedelic Storytelling

Muhammad Ibrahim, Mustafa Tariq, Connor Killingbeck, Khoa Nguyen

TABLE OF CONTENTS

01. PROJECT OVERVIEW
Enhancing wellness through AI-driven psychedelics research, promoting safe and informed experiences.

02. EXTERNAL DESIGN
NEUMA, is a chatbot for collecting psychedelic experiences, with simple user interaction and advanced administrative data management.

03. INTERNAL DESIGN
system's architecture, programming, data design, and API, focusing on simplicity and efficient component separation.

01.

HELLO!

In compliance with the Neuma Centre for Wellness, we're creating an adaptive chatbot to collect personal accounts of psychedelic experiences.

02.

EXTERNAL DESIGN

DESIGN PHILOSOPHY

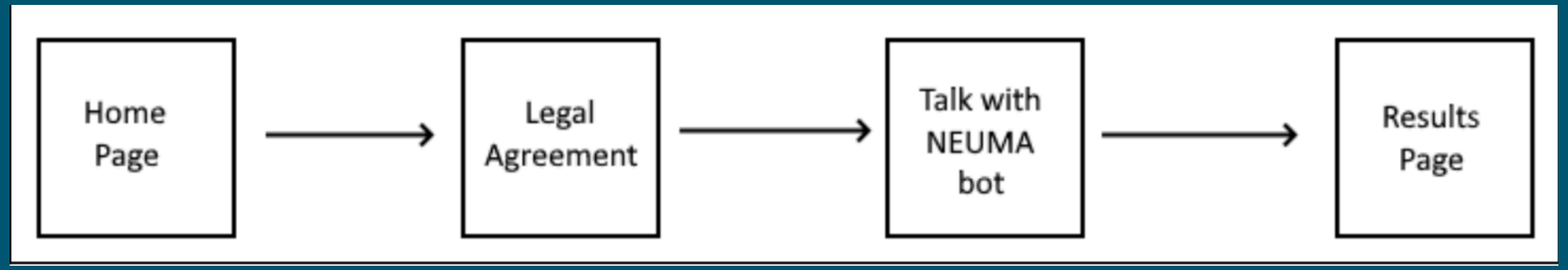
The goal of the User Interface Design was to cater to our two main groups of users. For contributors, users who will be speaking to the bot and giving their accounts of their experiences, we wanted to maintain a simple easy flow that did not challenge them technically.



Neuma

The Centre For Social Wellness

CONTRIBUTOR FLOW



NEUMA MINDSCAPE

LOGIN

Submit Your Experience!

TALK WITH NEUMA

Neuma Logo Here

About Neuma Center for Wellness

words words words

LEGAL MUMBO JUMBO

words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words
words words words words words words words words words words words

I ACCEPT

I DECLINE

NEUMA MINDSCAPE

LOGIN

Hi! My name is Neuma! I will be your report companion today!

ai logo



Sweet! I wanted to make a report about the crazy time I had on Mountain Dew last night

Okay! I would love to hear all the details! How did it all begin?

ai logo

Yeah I was with my friends and



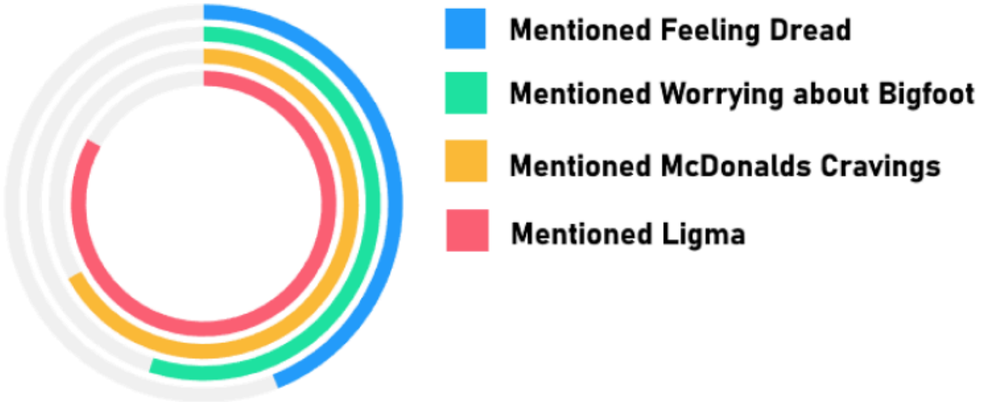
NEUMA MINDSCAPE

LOGIN

Thank you for your Contribution!

Below you will see your results!

TOPICS YOU COVERED



Share Your Results!

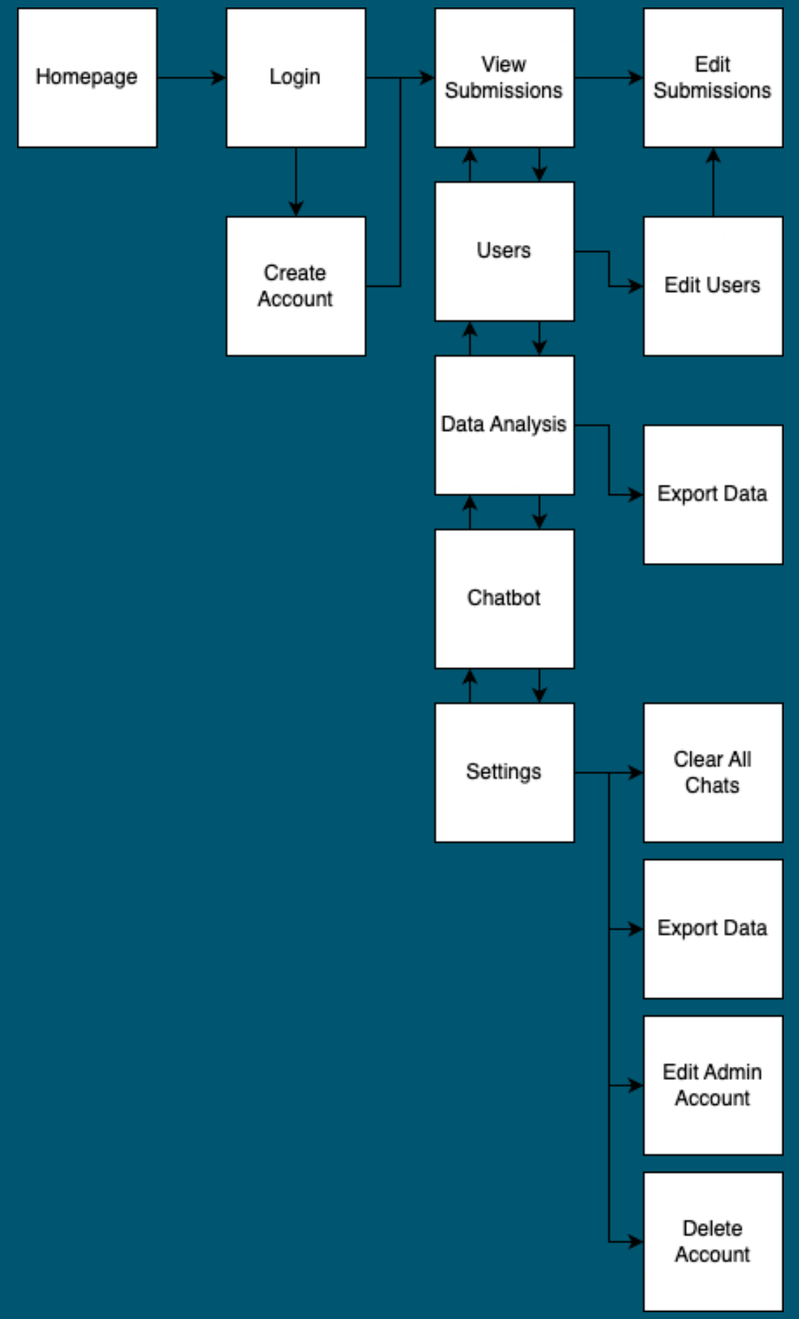
Download Image

Submit Feedback

ADMIN FLOW

Admins are logged in and are able to flexibly view submissions, users, data analysis, chatbots, and settings through the NAV bar.

Additional options for each section is available on each page, including account creation, deletion, and data exporting



NEUMA MINDSCAPE



NEUMA MINDSCAPE

Create Account

NAME: Mo Connor ROLE: Administrator



Biography:

Jane Doe, born in 1985 in Austin, Texas, is a renowned environmental scientist.



Reviews: 0

Posts: 0

Last Active: 01:05:42 - 2023/03/23

Submission ID	Date	More

N

help

View Submissions

Last Updated: 01:05:42 - 2023/03/23

View Submissions

Users

Data Analysis

Chatbot management

Settings

Search

Filter by: Default

Submission ID	Flag	Date	More
auvds_1342242_Adadf	+	01:05:42 - 2023/03/23	

RETURN TO LIST

Active	ID:	DATE:	SUBSTANCE:
Status	xxxxxxx	xx:xx:xx yyyy-yy-yy	xxxxxxxxxxxxx



FLAG FOR REVIEW

Complete Transcript:



words words words

words words words





words words words

N

help

View Submissions

Users

Data Analysis

Chatbot management

Settings

User Management

Last Updated: 01:05:42 - 2023/03/23

Search

Filter by: Default

	Username	Role	Date Created	Last Active	Options
	User1234	A	01:05:42 - 2023/03/23	01:05:42 - 2023/03/23	

RETURN TO LIST

NAME: Mo Connor ROLE: Administrator



Biography:

Jane Doe, born in 1985 in Austin, Texas, is a renowned environmental scientist.





Reviews: 7

Posts: 1

Last Active: 01:05:42 - 2023/03/23

Submission ID	Date	More
auvds_1342242_Adadf	01:05:42 - 2023/03/23	

N

help

Data Analysis and Reporting

- View Submissions
- Users
- Data Analysis
- Chatbot management
- Settings



Export Results

N

View Submissions

Users

Data Analysis

Chatbot management

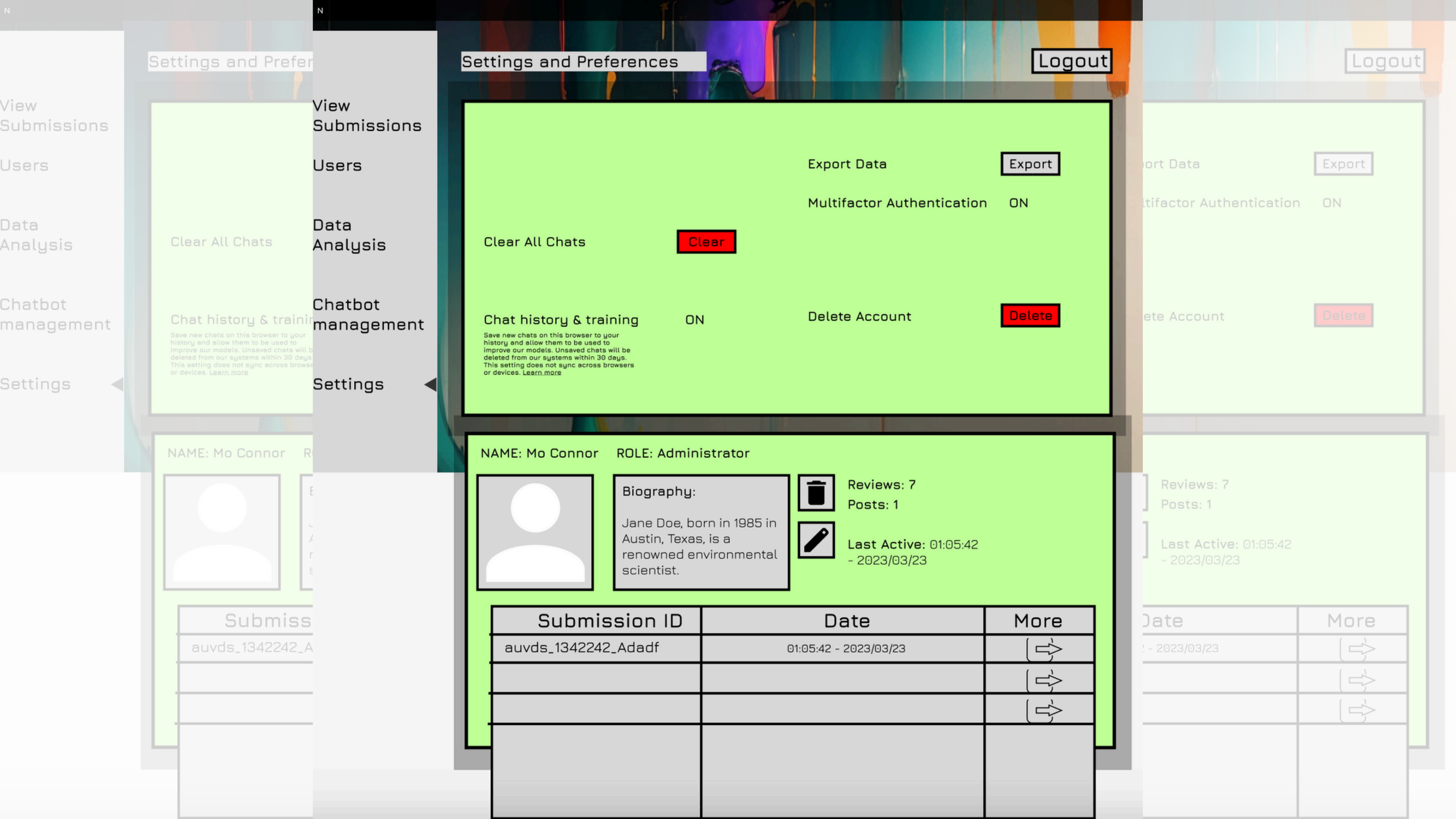
Settings

Chatbot Management



NAME: Charlie

VERSION: GPT 3.5 Turbo


Settings and Preferences
Logout
View Submissions
Users
Data Analysis
Chatbot management
Settings
Export Data
Export
Multifactor Authentication
ON
Clear All Chats
Clear
Chat history & training
ON
Save new chats on this browser to your history and allow them to be used to improve our models. Unsaved chats will be deleted from our systems within 30 days. This setting does not sync across browsers or devices. Learn more
Delete Account
Delete
NAME: Mo Connor
ROLE: Administrator
Biography:
Jane Doe, born in 1985 in Austin, Texas, is a renowned environmental scientist.
Reviews: 7
Posts: 1
Last Active: 01:05:42 - 2023/03/23
Submission ID
Date
More
auvds_1342242_Adadf
01:05:42 - 2023/03/23

CONLUSION

01. SIMPLICITY

The contributor section is user-friendly and not overly complex, with no incomplete reports filed if a user stops mid-way.

02. RESEARCHERS & ADMINISTRATORS

possessing higher technical skills, will have access to extensive settings, resources, and tools for data interaction.

03. DATA INPSECTION & PRUNING

Front-end tools will facilitate data inspection and pruning, while a shipped API will provide advanced functionality.

USAGE SCENARIOS:

CONTRIBUTORS

01. BROWSE

02. INCOMPLETE STORYTELLING

03. COMPLETED STORYTELLING BUT NO FEEDBACK

04. COMPLETED STORY WITH FEEDBACK



USAGE SCENARIOS:

RESEARCHERS & ADMINISTRATORS

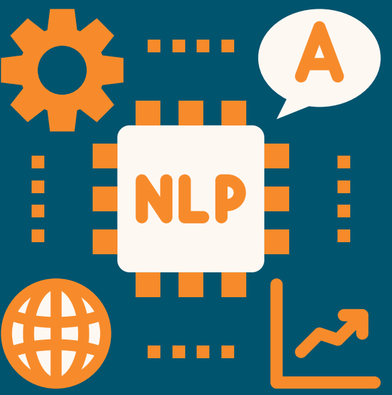
01. LOGIN TO ACCOUNT
02. VIEW SUBMISSIONS
03. EDIT SUBMISSIONS
04. MANAGE ACCOUNTS



Work products:

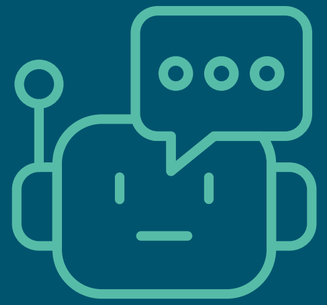
Leveraging Language Models

- Our project relies on Large Language Models.
- Enables user interaction and information gathering.
- Reports are sent for analysis and storage.
- Creates a self-fulfilling loop of data.
- Storage is managed through a database.



Work products:

Enhancing Chatbots



- Inputs include user emotion determination.
- Utilizes a built-in emotion recognition database.
- Requires specialized small databases.
- Examples: emotion detection, language patterns.

Work products:

Informative Visuals

- Result screen displays a radial bar chart.
- Charts shared experiences with contributors.
- Users can save, share, or ignore the chart.
- Chart generated once and saved locally by user choice.

INTEGRATION WITH EXISTING PROCESSES

SOLUTION

The project faces challenges in replacing the current system, which lacks visual consistency with the website, and the client wants to exclude existing reports but considers future data transfer.

Feedback From Customer



01. Simple Process for Contributors
02. Variety of Features Available to Researchers
03. A Love of the use of graphs and charts for the reward image
04. Changes to the visual aesthetic to match the current websites existing one
05. An expressed want for the usage of accounts, however a concern for the security issues this would bring
06. An expressed appreciation, but a further wanting of accessibility features

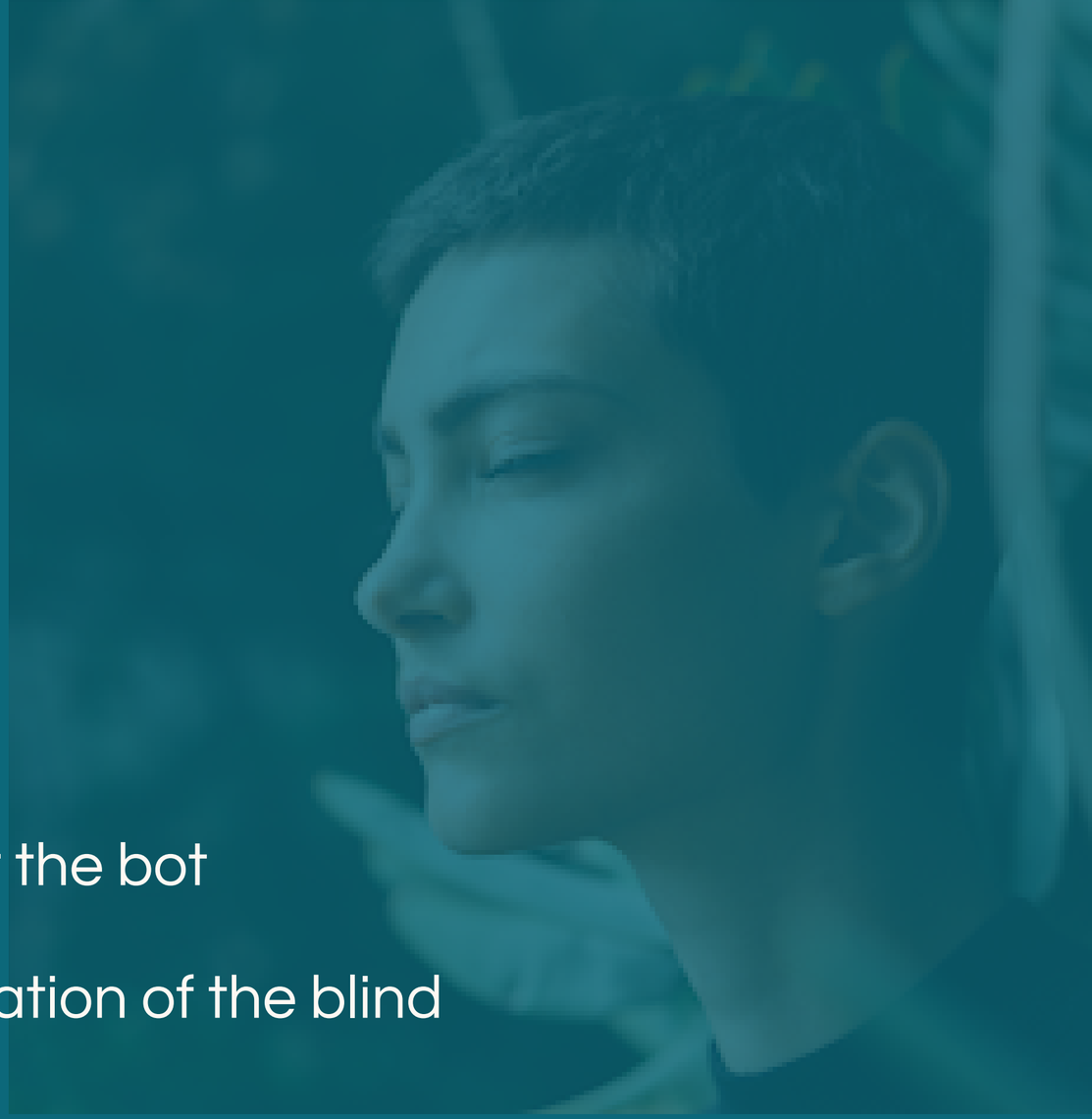
Feedback From Customer (continued...)

More accessibility features

01. Speech-to-text method

02 Text-to-speech compatibility for the bot

03. Proper metadata tags for navigation of the blind



EXTERNAL DESIGN DOC REQUIREMENTS UPDATE

01. More storage needed than originally anticipated
02. More systems than anticipated if no features get cut
03. Server load management and database optimisation
04. Different integration with websites than foreseen

BREAK!

03.

INTERNAL DESIGN

TABLE OF CONTENTS PT. 2

03.1 PROGRAMMING ENVIRONMENT
Tools necessary to build the system.

03.2 SOFTWARE ARCHITECTURE
Architectural style and design patterns.

03.3 DATA DESIGN
How we collect, organize and store data.

03.4 API DESIGN
Specifications of our API.

03.5 ALGORITHMS
How data is analyzed by the system.

03.6 NOTABLE TRADE OFFS
Trade offs to make to meet requirements.

03.7 NOTABLE RISKS
Detail plans to address and reduce risks.

03.8 MILESTONES
Updated milestones from our initial project plan.

03.1 PROGRAMMING ENVIRONMENT

PRACTICAL TOOL SELECTION APPROACH

01. DevTools
02 Languages
03. Web Stack
04. Data Analysis





Component View

The Neuma Mindscape system can be broken down into three major components: Frontend, Backend, and Database.



Deployment View

The Neuma Mindscape system is deployed on multiple nodes to ensure performance and availability. Here's how the components are allocated:

SOFTWARE ARCHITECTURE 3.2

COMPONENT VIEW

Frontend Components

User Interface

- User Registration and Login
- Chat Session List
- Chat Creation and Messaging



COMPONENT VIEW

Backend Components

Application Server

- User Management
- Chat Session Management
- Message Handling
- AI Response Handling



COMPONENT VIEW

Database

Data Storage



- Users Table
- Chats Table
- Messages Table

DEPLOYMENT VIEW

Frontend

- User interface components deployed on multiple frontend nodes (Front End 1 and Front End 2).
- Load balancing for even user traffic distribution.

DEPLOYMENT VIEW

Load Balancer

- Load balancer manages user requests.
- Ensures workload distribution and provides redundancy.

DEPLOYMENT VIEW

Backend

- Backend components handle requests and business logic.
- Deployed on backend servers (Backend Server 1 and Backend Server 2).
- Load balancing for request distribution and high availability.

DEPLOYMENT VIEW

Database

- Database stores user data, chat sessions, and messages.
- Typically hosted on a separate server or cloud service.
- Accessed by backend servers for data retrieval and storage.

CLASS-BASED VIEW

Key Classes

- User Class

  Attributes: userID, userType, email, password.
  Behaviors: register(), login()

- Chat Class

  Attributes: chatID, contributor.
  Behaviors: createChat(), listChats().

- Message Class

  Attributes: messageID, chat, messageText, timestamp.
  Behaviors: sendMessage(), listMessages().

- AIResponse Class

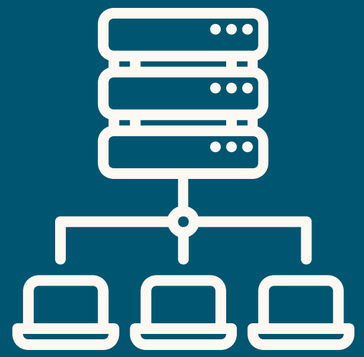
  Attributes: responseID, chat, responseText.
  Behaviors: sendAIResponse().

- AccessControl Class

  Attributes: None (it's a utility class).
  Behaviors: authorizeAccess().

- Database Manager Class

  Attributes: None (it's a utility class).
  Behaviors: insert(), update(), delete(), query().



Client-Server Architecture

The system follows a client-server architecture. The frontend (client) communicates with the backend (server) through API endpoints.



Three-Tier Architecture

The system is organized into three tiers - frontend, backend, and database.

ARCHITECTURAL STYLES AND DESIGN PATTERNS

ARCHITECTURAL STYLES AND DESIGN PATTERNS

Design Patterns

- Model-View-Controller (MVC)
- Repository Pattern
- JWT (JSON Web Token)



INTERFACES

Key Interfaces

- User Class Interface

register(user_type, email, password): Registers a new user with user type, email, and password.
login(email, password): Facilitates user login and returns a JWT token.

- Chat Class Interface

createChat(contributor): Creates a new chat session with a contributor and returns chatID.
listChats(user): Retrieves and returns a list of chat sessions associated with a user.

- Message Class Interface

sendMessage(chat, message_text): Sends a message in a chat session and returns messageID.
listMessages(chat): Retrieves and returns a list of user messages in a chat session.

- AIResponse Class Interface

sendAIResponse(chat, response_text): Sends an AI-generated response in a chat session and returns responseID.

03.3 DATA DESIGN

MVP SQL SCHEMA



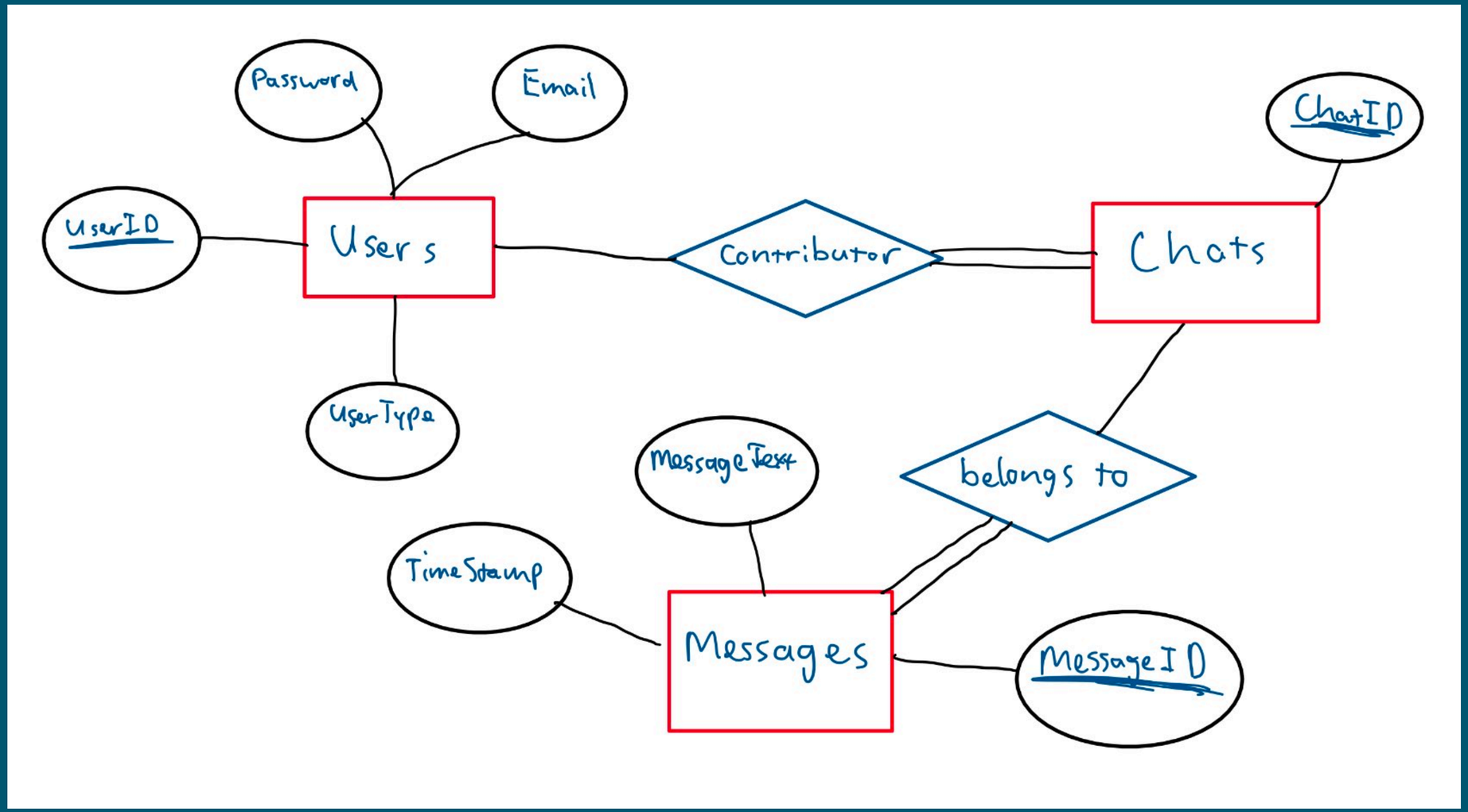

```
-- User table to store user information
CREATE TABLE Users (
    UserID INT AUTO_INCREMENT PRIMARY KEY,
    UserType ENUM('Administrator', 'Researcher', 'Contributor', 'Bot') NOT NULL,
    Email VARCHAR(255) NOT NULL UNIQUE,
    Password VARCHAR(255) NOT NULL
);

-- Chat table to store chat sessions
CREATE TABLE Chats (
    ChatID INT AUTO_INCREMENT PRIMARY KEY,
    ContributorID INT, -- Foreign key to Users table
    FOREIGN KEY (ContributorID) REFERENCES Users(UserID),
    CONSTRAINT chk_contributor_type CHECK (ContributorID IS NULL OR Users.UserType = 'Contributor')
);


-- Message table to store messages within a chat
CREATE TABLE Messages (
    MessageID INT AUTO_INCREMENT PRIMARY KEY,
    ChatID INT, -- Foreign key to Chats table
    MessageText TEXT NOT NULL,
    Timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (ChatID) REFERENCES Chats(ChatID)
);
```

03.3 DATA DESIGN

MVP SQL SCHEMA



03.4 API DESIGN

BACKEND REST API

AUTHENTICATION



- User Registration
  - Endpoint: PUT /api/users/register
  - Description: Allows contributors to register with the system.
  - Payload: user_type, email, password
  - Response: 200 OK on successful registration.

- User Login
  - Endpoint: POST /api/users/login
  - Description: Allows users to log in to their accounts.
  - Request Parameters: email, password
  - Response: 200 OK on successful login, returning a JWT token for authentication. 401 Unauthorized on login failure.

03.4 API DESIGN

BACKEND REST API

CHAT SESSIONS & MESSAGES



- Create Chat Session
- List Chat Sessions for a User
- Send Message

- Endpoint: POST /api/chats
- Description: Allows contributors to create a new chat session.
- Request Payload: contributor_id
- Response: 201 Created with the newly created chat session details.

- Endpoint: GET /api/chats/{contributor_id}
- Description: Retrieves a list of chat sessions associated with a user.
- Response: 200 OK with a list of chat sessions.

- Endpoint: PUT /api/messages
- Description: Allows users to send messages in an existing chat session.
- Request Payload: chat_id, message_text
- Response: 201 Created with the newly created user message details.

03.4 API DESIGN

BACKEND REST API

AI RESPONSES



- Send AI Response

- Endpoint: POST /api/ai-responses
- Description: Allows the AI system to send responses in a chat session.
- Request Parameters: chat_id, response_text
- Response: 201 Created with the newly created AI response details.

- List Messages (User and AI) in a Chat Session

- Endpoint: GET /api/messages/{chat_id}
- Description: Retrieves a list of user messages and AI responses in a specific chat session.
- Response: 200 OK with a list of user messages and AI responses in the chat session.

Access Control and Data Management:

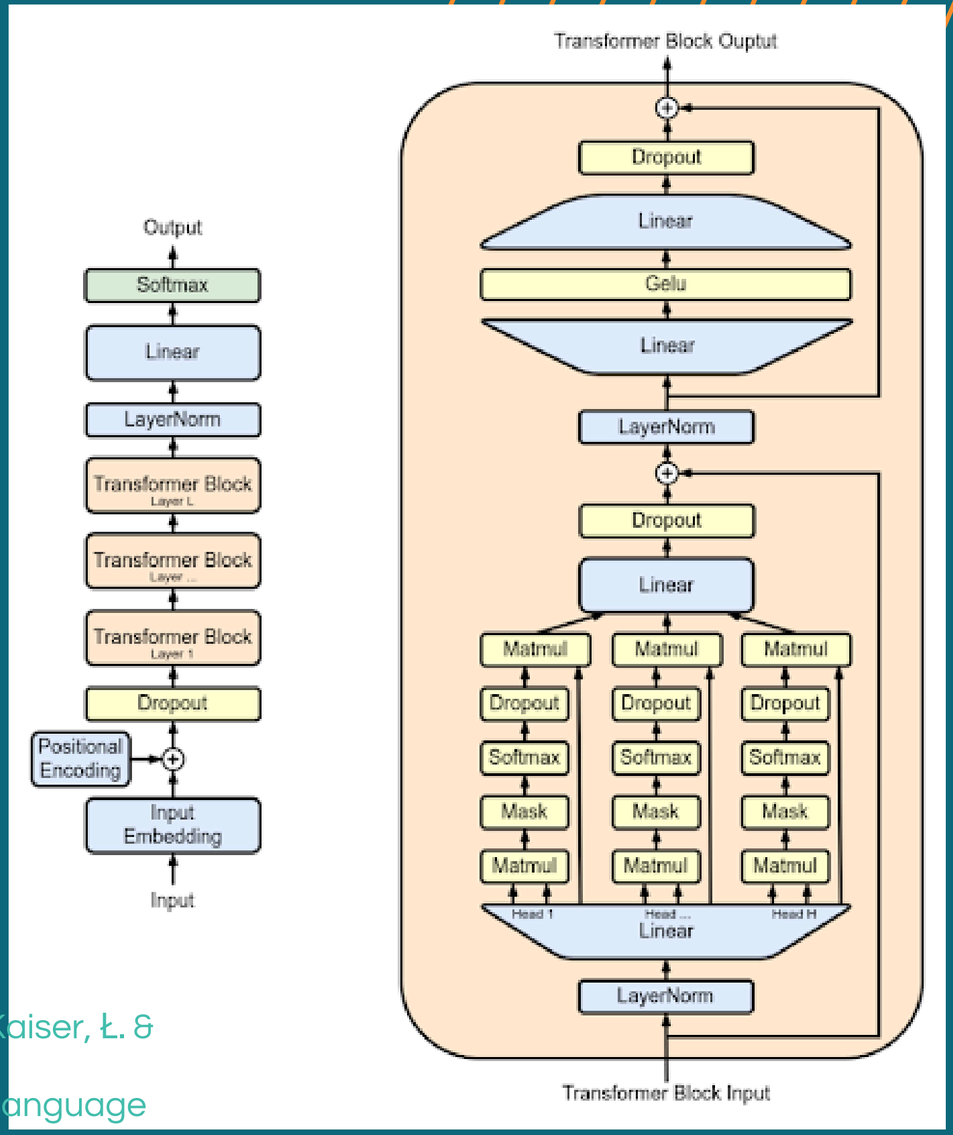
- Authentication
- Endpoint-Level Access Control
- Data Management
- Error Handling



03.5 ALGORITHMS

GPT & LLMs

- Transformer architecture
- Multi-head self-attention
- Feed-forward neural networks
- Pre-trained
- Valuable for NLP applications.



References:
Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł. & Polosukhin, I. (2017). "Attention is all you need."
Radford, A., Narasimhan, K., Salimans, T. & Sutskever, I. (2018). "Improving language understanding by generative pre-training."

03.5 ALGORITHMS

How to Effectively Use LLMs

STRATEGIES

- User-Centered Design
- Human-in-the-Loop
- Feedback Mechanisms
- Testing and Evaluation
- Scalability

- **Creating a Chatbot**
  User engagement and preferences.
- **Fine-Tuning**
  Enhancing smaller language models for specialized tasks.

03.6 NOTABLE TRADE OFFS

- Security vs. User Convenience
- Performance vs. Scalability
- Data Storage vs. Efficiency
- User Experience vs. Security
- Flexibility vs. Complexity
- Latency vs. Consistency
- Complexity vs. Extensibility



03.7 NOTABLE RISKS

Identifying Potential Risks

KEY AREAS OF CONCERN IN THE INTERNAL DESIGN.

- Security Risks

  Risk: Storing user passwords securely.
  Mitigation: Implement strong encryption and hashing, and update security measures regularly.

- Scalability and Performance Risks

  Risk: Potential performance issues with increased user load.
  Mitigation: Monitor and optimize system performance, consider database migration for scalability.

- Complexity and Maintainability Risks

  Risk: Complexity due to design patterns.
  Mitigation: Ensure comprehensive documentation, coding standards, and regular code reviews.

- Data Integrity Risks

  Risk: Data consistency in multi-node deployments.
  Mitigation: Implement strategies like distributed databases or data replication.

03.7 NOTABLE RISKS

Identifying Potential Risks (continued...)



KEY AREAS OF CONCERN IN THE INTERNAL DESIGN.

- **User Experience Risks**

  Risk: Balancing data security and user satisfaction.
  Mitigation: Gather user feedback, enhance error messages, and improve processes.

- **Database Scalability Risks**

  Risk: SQLite limitations as the system grows.
  Mitigation: Consider migrating to a more scalable database solution.

- **Security Token Risks**

  Risk: JWT token security.
  Mitigation: Employ best practices for securing JWT tokens.

- **Access Control Risks**

  Risk: Correct implementation of access control rules.
  Mitigation: Regularly audit, test, and strengthen access control mechanisms.

- **Concurrency Risks**

  Risk: Managing simultaneous access within chat sessions.
  Mitigation: Implement concurrency control mechanisms, such as locking or optimistic concurrency control.

03.8 MILESTONES

Date	Milestone	Description
Nov 27	Prototype	Completion of mockup prototype product.
Nov 27	Design Document	Description of the system, completion of the user interface design, object models, module structural dependencies, architecture decided, data structures and algorithms determined.
Winter Break		
Jan 11	Setup Dev Environment	Get all team members on the same page regarding the programming environment. GitHub repo created, packages and all prereqs installed. Take inspiration from design document.
Jan 18	Processes and Metrics Explored	All team members discuss personal views on important activities and metrics for quality assurance. Final choices made.
Jan 25	Document Rough Draft Created	Speak to supervisor about changes that need to be made and high level suggestions before final submission.
Jan 29	Quality Assurance and Deployment Plan	Identification of software process activities and metrics, description of assurance activities, descriptions of documents produced, detailed plans for the delivery and deployment.
Feb 1	Code Skeleton	Create functions, objects, and setup documentation generators for projects so comments in code are generated into web pages. As well as an installation guide.
Feb 8	Assign Tickets in GitHub Projects	Assign tickets to each team member so everyone has a list of tasks to get done to complete the project, goal being to move as fast as possible while sticking to our processes.
Reading Week		
Feb 29	Initial Customer Showcase	Showcase half baked system to customer, discuss incomplete items and items that may not be completed by the due date. Prioritize features to ensure maximum customer satisfaction.
Mar 14	Implement Feedback	By this date we are secure in what we've accomplished, we may omit some features or include some others, but this is what we are able to accomplish. Now prioritize code review, testing, and eliminating bugs.
Mar 21	Final Customer Showcase	Last showcase of the system to the customer, here they can request for any last second changes before handoff.
Mar 25	Project Completed	Official handoff of system to customer.
Mar 25	Final Project Documentation	User manual, installation guide, maintenance suggestions, known limitations, workarounds, updates to system internal designs.

“JUST KEEP MOVING FORWARD.”

THANK YOU

Have any questions?